

Evolution of the GΦL Workflow Application

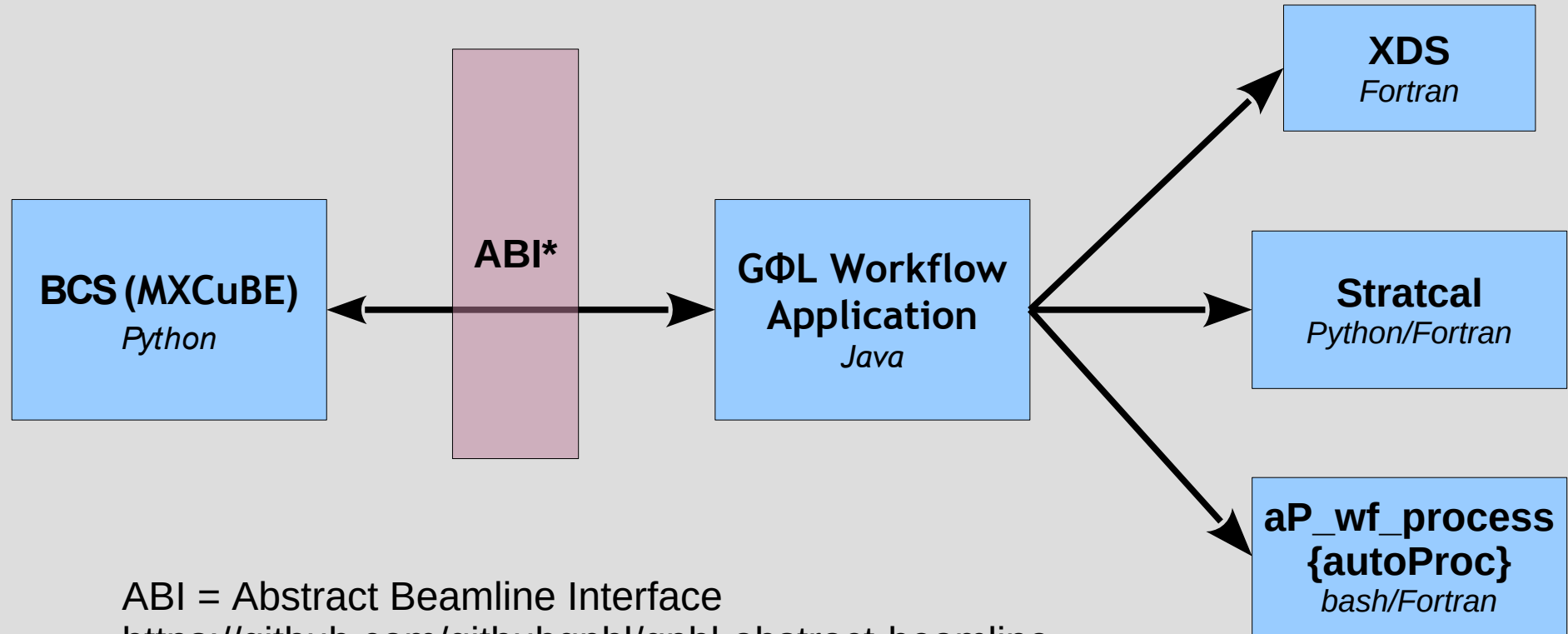
Peter Keller
Global Phasing Ltd.

Contents

- Issues connected with/arising from deploying the GΦL workflow for testing and production
 - Issues raised by beamline scientists and users
 - Quirks of real samples and instrumentation
- History, technical debt and avoiding obsolescence

The workflow has a long history. I will try to reassure beamline personnel deploying it that they won't end up wrestling with a lot of historical baggage.

GΦL Workflow Overview



ABI = Abstract Beamline Interface
<https://github.com/githubgphl/gphl-abstract-beamline>

Towards production deployment

- Semantic versioning of the G Φ L Workflow Application
 - <https://semver.org>
 - Version info provided to MXCuBE via the ABI
 - MXCuBE checks workflow version at runtime
 - Patching/bug-fixing of deployed versions
 - Either workflow application or whole workflow package
 - Controlled introduction of new versions
 - Automatic building of multiple versions almost ready (Clemens Vonnrhein)
 - Current versions
 - 2.1.x (ESRF/MASSIF-1, SOLEIL/PX2, MAX-IV/BioMAX)
 - 2.2.x (DESY/P14)
- Increasing WF use => more multi-orientation datasets

Contact with the real world

- Some features that have been recently introduced in response to issues arising (mostly from P14)
 - User-provided reference reflection files
 - Passed to autoPROC
 - Mosaicity estimation from characterisation data collection
 - Selection of image width for main data collection
 - Acceptable image widths are configured in MXCuBE
 - Starting estimate for processing of problematic (low-mosaicity) samples

Contact with the real world - symmetry (1)

- Enhanced symmetry handling
 - Currently applicable mostly to attended mode

Symmetry/cell input

LATTICE- CHARACTER		BRAVAIS- LATTICE	QUALITY OF FIT	UNIT CELL CONSTANTS (ANGSTROM & DEGREES)					
				a	b	c	alpha	beta	gamma
*	44	aP	0.0	56.4	56.4	102.5	90.0	90.0	90.0
*	31	aP	0.0	56.4	56.4	102.5	90.0	90.0	90.0
*	33	mP	0.0	56.4	56.4	102.5	90.0	90.0	90.0
*	35	mP	0.1	56.4	56.4	102.5	90.0	90.0	90.0
*	34	mP	0.1	56.4	102.5	56.4	90.0	90.0	90.0
*	32	oP	0.1	56.4	56.4	102.5	90.0	90.0	90.0
*	14	mC	0.2	79.7	79.7	102.5	90.0	90.0	90.0
*	10	mC	0.2	79.7	79.7	102.5	90.0	90.0	90.0
*	13	oC	0.2	79.7	79.7	102.5	90.0	90.0	90.0
*	11	tP	0.2	56.4	56.4	102.5	90.0	90.0	90.0
	37	mC	249.9	212.6	56.4	56.4	90.0	90.0	74.6
	36	oC	249.9	56.4	212.6	56.4	90.0	90.0	105.4
	28	mC	249.9	56.4	212.6	56.4	90.0	90.0	74.6

Contact with the real world - symmetry (2)

- Enhanced symmetry handling
 - Currently applicable mostly to attended mode
 - Selection of indexing solution/Bravais lattice not always enough
 - Properties of particular samples can fool indexing software into assigning the wrong Bravais lattice
 - If the correct combination of unit cell and lattice does not appear in the output of XDS(IDXREF), user can tweak symmetry information
 - Change lattice centring mode; adjust point group.
 - Only partially tested

Bravais lattices and space groups

***** LATTICE SYMMETRY IMPLICATED BY SPACE GROUP SYMMETRY *****

BRAVAIS- TYPE	POSSIBLE SPACE-GROUPS FOR PROTEIN CRYSTALS [SPACE GROUP NUMBER,SYMBOL]
aP	[1,P1]
mP	[3,P2] [4,P2(1)]
mC,mI	[5,C2]
oP	[16,P222] [17,P222(1)] [18,P2(1)2(1)2] [19,P2(1)2(1)2(1)]
oC	[21,C222] [20,C222(1)]
oF	[22,F222]
oI	[23,I222] [24,I2(1)2(1)2(1)]
tP	[75,P4] [76,P4(1)] [77,P4(2)] [78,P4(3)] [89,P422] [90,P42(1)2] [91,P4(1)22] [92,P4(1)2(1)2] [93,P4(2)22] [94,P4(2)2(1)2] [95,P4(3)22] [96,P4(3)2(1)2]
tI	[79,I4] [80,I4(1)] [97,I422] [98,I4(1)22]
hP	[143,P3] [144,P3(1)] [145,P3(2)] [149,P312] [150,P321] [151,P3(1)12] [152,P3(1)21] [153,P3(2)12] [154,P3(2)21] [168,P6] [169,P6(1)] [170,P6(5)] [171,P6(2)] [172,P6(4)] [173,P6(3)] [177,P622] [178,P6(1)22] [179,P6(5)22] [180,P6(2)22] [181,P6(4)22] [182,P6(3)22]
hR	[146,R3] [155,R32]
cP	[195,P23] [198,P2(1)3] [207,P432] [208,P4(2)32] [212,P4(3)32] [213,P4(1)32]
cF	[196,F23] [209,F432] [210,F4(1)32]
cI	[197,I23] [199,I2(1)3] [211,I432] [214,I4(1)32]

Classification of symmetry



teaching and education



JOURNAL OF
APPLIED
CRYSTALLOGRAPHY

ISSN 1600-5767

Crystallographic shelves: space-group hierarchy explained

Massimo Nespolo,^{a*} Mois Ilia Aroyo^b and Bernd Souvignier^c

^aUniversité de Lorraine, CNRS, CRM2, Nancy, France, ^bFísica de la Materia Condensada, Facultad de Ciencia y Tecnología, Universidad del País Vasco, Apartado 644, Bilbao 48080, Spain, and ^cInstitute for Mathematics, Astrophysics and Particle Physics, Faculty of Science, Mathematics and Computing Science, Radboud University Nijmegen, Postbus 9010, Nijmegen 6500 GL, The Netherlands. *Correspondence e-mail: massimo.nespolo@univ-lorraine.fr

Received 19 May 2018

Accepted 7 September 2018

Edited by G. J. McIntyre, Australian Centre for Neutron Scattering, Australia

Keywords: space-group classification; crystal classes; crystal systems; crystal families; Bravais classes; lattice systems.

Space groups are classified, according to different criteria, into types, classes, systems and families. Depending on the specific research topic, some of these concepts will be more relevant to the everyday crystallographer than others. Unfortunately, incorrect use of the classification terms often leads to misunderstandings. This article presents the rationale behind the different classification levels.

Arithmetic crystal classes (1)

- There are 24 arithmetic crystal classes compatible with protein structures, comprising 65 space groups
 - They contain all the symmetry information required for data processing
 - Formed by combining point group and lattice centring
- Compact representation: $[4P, 422P]$ equivalent to:

tP $[75, P4]$ $[76, P4(1)]$ $[77, P4(2)]$ $[78, P4(3)]$ $[89, P422]$ $[90, P42(1)2]$
 $[91, P4(1)22]$ $[92, P4(1)2(1)2]$ $[93, P4(2)22]$ $[94, P4(2)2(1)2]$
 $[95, P4(3)22]$ $[96, P4(3)2(1)2]$

- Contain all required information for strategy calculation and data processing

Arithmetic crystal classes (2)

- Good for representing multiple possible symmetries without forcing premature decision-making
 - Sometimes, selection of the final space group has to be postponed to downstream structure solution or refinement anyway
- Symmetry-related UI choices are translated into arithmetic crystal classes behind the scenes
 - Users are not exposed to them directly - they are not as widely known as space groups.

Definitely on the TODO list

- Specification of autoPROC macros to workflow
- Comparison of unit cell parameters
 - For reliable selection of indexing solution in unattended mode
- Visualisation of characterisation/indexing
- Resolution estimation from Dozor for strategy calculation
 - Will need input from individual beamlines: Dozor deployments seem to be highly divergent
 - What happens when detector is too far away?
- Choices after strategy calculation
 - loop back to indexing solution rather than continuing with data collection
- Technology change for Java/Python communication
 - Py4J is limiting and not problem free

Under consideration

- Multiple known crystal forms
 - Particularly important for unattended operation
 - Will LIMS systems ever support this?
 - “We never specify the crystal form, because being restricted to providing just one does more damage than providing none”
- Small molecules?
 - Experience with very high resolution protein work (up to 0.43Å) does not necessarily transfer to small molecules
 - Catering for all possible crystallographic symmetry:
 - 73 arithmetic crystal classes; 230 space groups
 - Characterisation/indexing for small unit cells
 - => sparse diffraction pattern
 - Impact on strategy calculation and processing is unknown

Contents

- Issues connected with/arising from deploying the GΦL workflow for testing and production
 - Issues raised by beamline scientists and users
 - Quirks of real samples and instrumentation
- History, technical debt and avoiding obsolescence

The workflow has a long history. I will try to reassure beamline personnel deploying it that they won't end up wrestling with a lot of historical baggage.

Keeping an application current

- Matias Guijarro & “technical debt”:
MXCuBE’s earlier dependence on Qt3
- Over time, technical debt and creeping obsolescence can become crippling, inhibiting further development and compromising deployment
 - Languages and their standard libraries/APIs evolve
 - Third-party dependencies
 - Tooling (IDEs, build tools)

Java evolution (1)

- The Java release cadence is quick, but manageable:
 - Some releases are LTS; the latest one is 25 (released 2025-09)
 - Newer versions of Java can target older versions when compiling, and can execute older bytecode.
 - Not a panacea:
 - A few public APIs have been withdrawn
 - Tightening up of questionable practices (e.g. use of non-public APIs) can also cause issues with old code
 - Older versions of Java are still readily available, and are still being built (even for newer architectures such as aarch64) and patched
 - <https://adoptium.net> and others
 - The earliest historical version of Java that is a practical option is probably Java 8 LTS (released 2014). Its latest patch is update 492, released 2026-05

Java evolution (2)

- The GΦL workflow application started with Java 5
 - Some components upgraded bit-by-bit to Java 6, then Java 7
- 2014: Java 8 LTS released
 - Introduced functional programming-style constructs
 - lambdas, references to methods, streams
 - Workflow migrated in stages over an extended period of time.
Final upgrade of all components to Java 8 achieved around 2020
 - Theoretically, I could have settled on this version permanently.

Java evolution (3)

- 2018: Java 11 LTS released
 - Java Platform Module System (JPMS, aka Project Jigsaw)
 - An application is divided up into modules
 - In physical terms a module is a special type of JAR file.
 - Access to one module's resources by another module is tightly specified
 - Workflow migrated to Java 11 2023-10
 - Made fully modular 2024-04
 - The process entailed a lot of cleanup and useful refactoring, with remaining untidiness well encapsulated

Java evolution (4)

- Java versions > 11 have some “nice to have” new features, but for our purposes nothing compelling.
- May migrate in the future if something else comes up
- With the JPMS, it becomes easier to create self-contained applications
 - This would allow the use of a recent Java version for the workflow, without requiring a beamline to provide a compatible Java installation.

Technical debt: build system (1)

- Initially used a custom Python application to build the workflow, with extensive use of shell scripts
- Workflow started in collaboration with DLS. The Beamline Control System was GDA
 - GDA was an Eclipse/PDE application
 - PDE: Plug-in Development Environment
- PDE tooling is easily accessible in the Eclipse IDE
 - Provided tempting short-cuts, but created technical debt

Technical debt: build system (2)

- Moved to an ANT-based build, using Eclipse/PDE
 - ANT build files initially generated by Eclipse, then modified by hand
 - The workflow was never distributed with any Eclipse components.
 - Still depended on some shell scripts
- Managing 3rd-party dependencies was becoming painful.
- Eclipse/PDE was becoming a crutch

Technical debt: build system (3)

- For maintainability, building distributions without using Eclipse was becoming vital
 - Settled on Gradle: <https://gradle.org/>
- Gradle, like Java, has a rapid release cadence.
 - Major upgrades to Gradle often bring clear and worthwhile improvements.
 - Keeping up brings increased robustness and maintainability
 - Enforcement of good practice
 - Migration from Groovy to Kotlin (dynamically and statically typed respectively)
 - Exploration of Gradle started with version 4.x, then migrated to 6.8.3, 7.4.1, 8.5, 8.14.1 and is now about to move to 9.5.1
 - Migration to JPMS happened in the context of Gradle 8.5

Technical debt: build system (4)

- Gradle benefits include:
 - Robust and clear dependency management
 - Useful management of 3rd-party dependencies
 - Initially as a collection of downloaded JARs in a directory, later as a Maven-format repository (hosted internally on our LAN)
 - Useful analysis of the build
 - Gradle's output of the dependency tree was what enabled me to finally throw away the Eclipse/PDE crutch
 - Robust diagnostics
 - Warnings/errors particularly good after migrating from Groovy to Kotlin.
 - Improvements as Gradle evolves.

Technical debt: build system (5)

- Gradle has a steep learning curve
 - Much easier to start a new project than migrate an existing one
 - Still a lot better than learning Maven
 - Dealing with friction generally results in a clean and robust outcome
 - Hacky workarounds don't accumulate
 - With ANT, it was a very different story

Technical debt: misc

- Technical debt avoided: CVS → git
 - In 2012: Thanks to Claus Flensburg for taking the lead in introducing git to Global Phasing.
 - Much of the work described in this presentation would have been hard or impossible with CVS
- Papyrus, Xtend/Xpand (*not* Xtend2) and XMLBeans
 - UML modelling and model transformation
 - This is the last aspect that still requires Eclipse/PDE
 - No direct impact on workflow deployment
 - XML API generation from XSD
 - True technical debt, but for now remains manageable
 - Any migration away from these would probably need to be AI-assisted

Acknowledgements

- GΦL: Gérard Bricogne, Claus Flensburg, Rasmus Fogh, Clemens Vornrhein
- ESRF: Matthew Bowler, Olof Svensson
- EMBL-Hamburg: Gleb Bourenkov
- MPI: Ashwin Chari
- MAX IV: Ana Gonzalez, Zac Panepucci
- SOLEIL: Martin Savko, Bill Shepard